Practica05_Montes_Peréz

Daniel Montes Peréz 323077552

October 2026

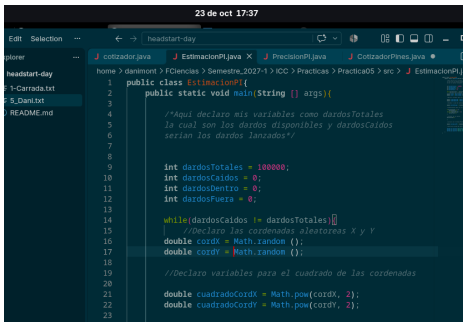
1 Numero π con dardos

1.1 Aproximación a π

en esta parte nos pidieron realizar un código que calcule el numero pi al lanzar dardos aleatorios en un cuadrado de 1x1 y con un cuarto de circulo para realizar una aproximación a π entonces lo primero que hice es declarar mis variables con el numero de dardos y las demás con valor 0 para que se vayan actualizando en el código.



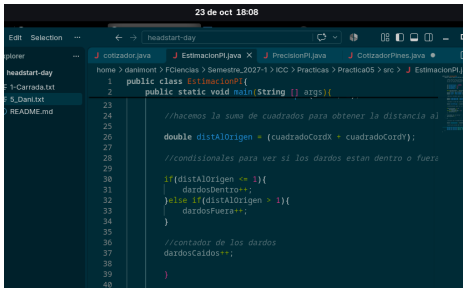
Mi siguiente paso fue utilizar un while para que se hagan los lanzamientos aleatorios de los dardos, con la condición que el valor de los dardos que caigan sean diferentes al valor de los dardos totales que tenemos para que el ciclo pueda finalizar, una vez dentro del while declaramos dos nuevas variables y les asignamos valores aleatorios con **Math.random** para obtener las dos coordenadas del dardo, seguido de eso de nuevo declaramos dos variables para guardar el cuadrado de cada una de las coordenadas las cuales elevamos con **Math.pow** .



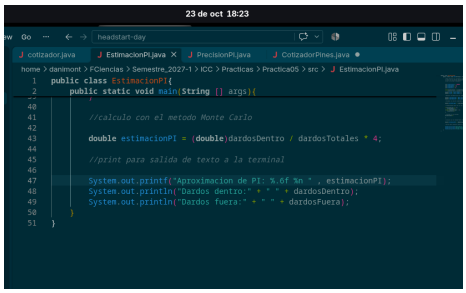
Mi siguiente paso declarar una nueva variable para realizar el calculo de la distancia entre el centro del circulo y el dardo para saber si cayó dentro o fuera de el con la siguiente formula

$$x^2 + y^2 \leq 1$$

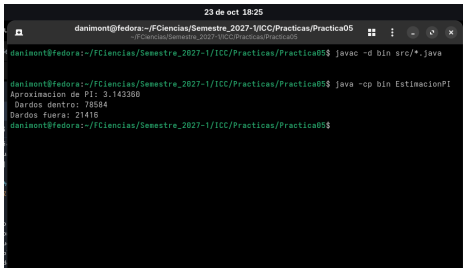
lo cual si es menor o igual que 1 esta dentro y si es mayor esta fuera, sabiendo esto utilice un if para hacer una condición de que si el valor de la variable *distAlOrigen* es $\leq$ se sumara 1 en la variable *dardosDentro* que declare al inicio con valor 0, debajo agregue un else if con la condición de que la misma variable sea $\geq$ que 1 y si llo cumple que le sume 1 a la variable *dardosFuera.* saliendo del if y el else if agrege una suma de 1 a la variable *dardosCaidos* la cual tiene la función de ser el contador de los dardos que lanzamos para que el while tenga fin.

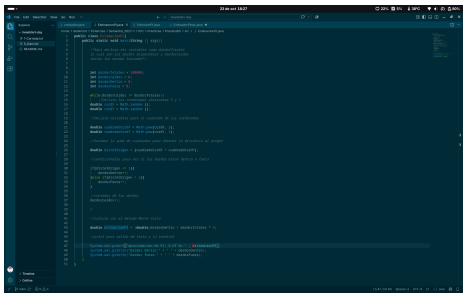


Una vez fuera del while aplicamos el método Monte Carlo y utilizamos pintf para indicar en la consola el total de dardos dentro y fuera del circulo ademas de la aproximación a la que llegamos con la formula.



Aquí pongo imágenes de como funciona con 100000 y del código completo.





Realizando los experimentos que nos pidieron me arrojaron estos resultados:

Num. de experimento	Num. de dardos	Apraximación de π
1	100	3.160000
2	100	3.000000
3	100	2.920000
4	1000	3.188000
5	1000	3.200000
6	1000	3.132000
7	10000	3.119600
8	10000	3.157600
9	10000	3.147200
10	100000	3.141840
11	100000	3.146440
12	100000	3.134160
13	1000000	3.141848
14	1000000	3.139888
15	1000000	3.139384

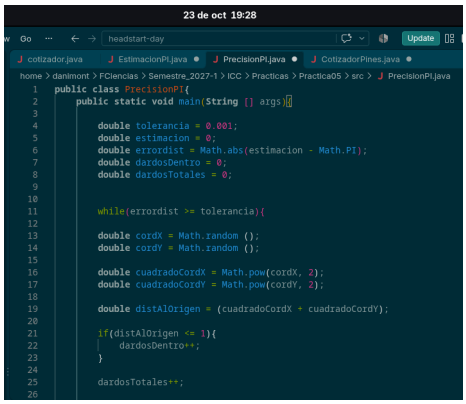
En conclusión lo que yo noto es que la aproximación va siendo cada vez mas exacta al aumentar el numero de dardos que se lanzan y el experimento no da valores distintos de aproximación con el mismo valor de dardos totales porque al ser valores aleatorios no siempre van a caer el mismo numero de dardos adentro del circulo.

1.2 Precisión π

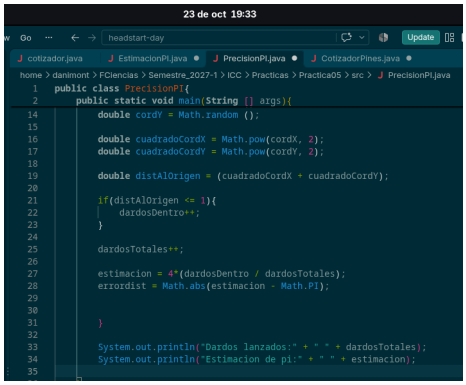
Para esta sección de la practica nos pidieron algo parecido a el ejercicio anterior, por lo cual utilice de base el código de antes, solo agregando las variables *tolerancia* y *errorDist* las cuales con la tolerancia y la distancia entre la aproximación de pi y pi, lo cual al ser distancia necesita ser siempre positivo, entonces utilizamos **Math.abs**



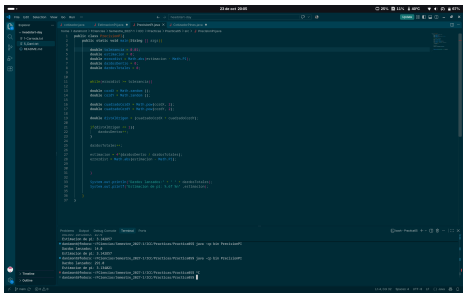
Aquí utilizamos de nuevo while para lanzar nuestros dardos aleatorios exactamente igual al ejercicio anterior elevamos al cuadrado las coordenadas y utilizamos la formula para obtener su distancia al origen, mantenemos el if con la condición *distAlOrigen*≤ 1 y si la cumple se le suma 1 a la variable *dardosDentro*, ademas de seguir teniendo el contador de dardos lanzados sumándole 1 a la variable *dardosCaidos*.

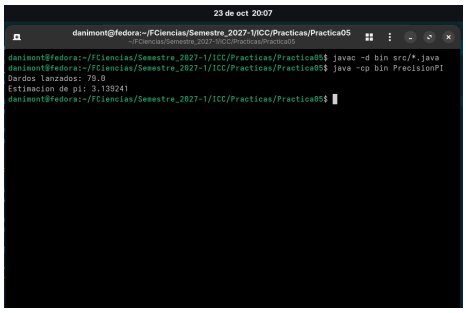


De aquí lo diferente al ejercicio anterior esta en la condición del while aquí nos pidieron que el ciclo siga mientras $errorDist \geq tolerancia$ y dentro del while agregamos la formula de Monte Carlo para ir calculando la aproximación de pi, la cual utilizamos para calcular la actualización del while la cual es la variable *errorDist* y esta va ir calculando la distancia entre nuestra aproximación de pi y el valor real de pi, así al llegar al valor de nuestra tolerancia de error la condición hará que salga del while, al final agregue dos print para que muestre el valor de la aproximación de pi y los dardos que se lanzaron.



Aquí agrego fotos de la ejecución y del código completo, agrego que la mayor parte de esta seccion de la practica la hice en el laboratorio con ayuda de Victor en algunas cosas.





Al hacer los experimentos que nos pidieron estos fueron los resultados:

Num. de experimento	Tolerencia	Numero de dardos utilizados	Aproximacion de PI
1	0.1	66	3.212121
2	0.1	821	3.239951
3	0.1	5	3.200000
		Promedio: 297	
4	0.01	28	3.142857
5	0.01	14	3.142857
6	0.01	291	3.134021
		Promedio: 111	
7	0.001	24613	3.142567
8	0.001	937	3.141942
9	0.001	247	3.141700
		Promedio: 8599	

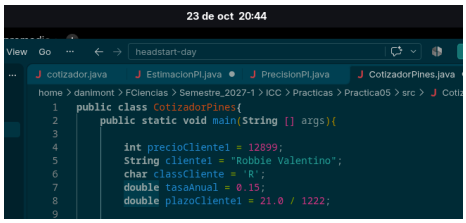
En conclusión si variaron bastante algunos en el numero de dardos necesarios para llegar al resultado, por ejemplo en el de tolerancia 0.1 en un ejercicio se utilizaron 5 y en otro 821, en comparación al ejercicio anterior la diferencia radica en que el programa finaliza cuando la aproximación llega a la tolerancia que dimos y en el otro caso damos los dardos que queremos lanzar sin importar si esto hace mas inexacto el calculo. osea uno solo utiliza los necesarios y el otro hace el calculo con los dardos impuestos



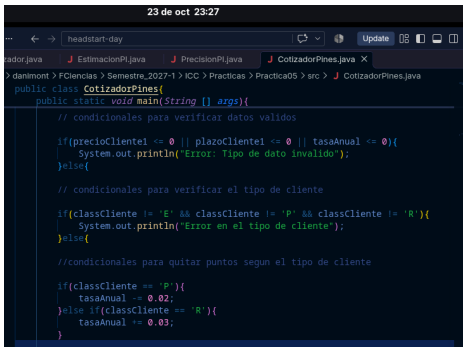
Figure 1: Error en el tipo de cliente

2 Cotizador Pines

Aquí empece con la base del primer programa que hicimos en la practica 04 asi que hasta aqui son las mismas variables



En esta parte añadí las condicionales para detectar los datos incorrectos, primero con un if agregue una condición de que los valores como el plazo a meses, el valor de préstamo y la tasa anual con una disyunción lógica en donde puse que los valores sean menores a 0 salta un mensaje que dice "tipo de dato no valido", después puse un else y si no se cumple esta condición se seguirá ejecutando el código, seguido agregue otro if para verificar el tipo de cliente ahora con una conjunción lógica la cual hará que si no se cumple alguna de las condiciones de valor false y imprimirá el mensaje "Error en el tipo de cliente" en acaso de no cumplirse ira al else y seguirá el código. Después hay un ultimo if que sirve para agregar o quitar puntos a la tasa anual segun el tipo de cliente.



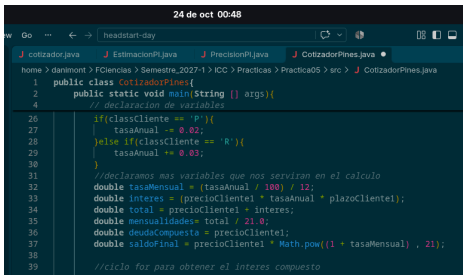
Aquí pongo imágenes de que pasa cuando fallan los tipos de datos



Figure 2: Error por precio negativo

2.1 Interés compuesto

Después vienen las variables de calculo, que son las mismas que utilice en la practica 04 para el cotizador, las únicas nuevas son *tasaMensual*, *deudaCompuesta* y *saldoFinal* donde calculo la tasa mensual para poder obtener el interés compuesto declaro la variable deuda compuesta para utilizarla en el ciclo for y la variable saldo final para verificar que todo salga bien, utilizando la formula que nos proporcionaron.

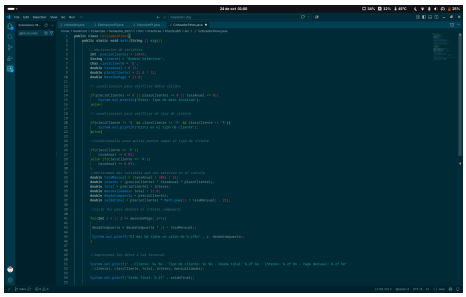


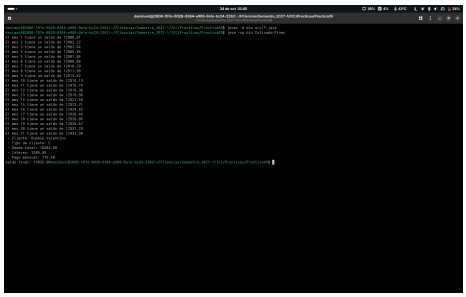
Segudo de esto hacemos nuestro siclo for, con una variable iniciada en 1 y una condición que pare cuando dicha variable sea mayor a el plazo de meses en el que se pagara, teniendo una actualización de suma 1 cada ciclo, y en cada ciclo imprime el mensaje "El mes (numero de mes) tiene un saldo de (saldo del mes).

la ultima parte unicamente imprime los valores que ya estaban calculados desde la practica 04, el ultimo print sirve para verificar que el calculo haya sido correcto con el ciclo for



Adjunto imagen del codigo completo y de como se ejecuta





3 Conclusión

En mi conclusión agregaría que se me hizo algo sencillo y entretenido de hacer sobre todo el hecho de plasmar lo visto en clase de manera practica, disfruto mucho el poder decir que entiendo lo que estoy haciendo, ademas les veo muchas aplicaciones a las condiciones con if y else y los ciclos con while y for.